Test-Time Preference Optimization in an Enterprise Optimization Feedback Loop

Final report on Tasks 1 to 5

Yuqian Zhang

28 October 2026

Abstract. This report reproduces Test-Time Preference Optimization (TPO) on a public benchmark and then applies it to a feedback loop around a small vehicle routing model. On 100 AlpacaEval 2 prompts with an 8B supervised fine-tuned policy, the run under a frozen protocol raised the judge win rate from 8.77% to 12.09% and 14.13% over two TPO iterations. The two-step gain is reliable, but the length-controlled win rate did not rise, and TPO did not beat Best-of-15 sampling with the same number of candidates. For the routing problem, three components explain a plan, answer what-if questions and localize vague complaints. On ten test cases with brute-force ground truth, attribution was always correct and localization was correct in 90% of first passes, with every low-confidence case turned into a question to the user. An ablation with 550 trials found, with moderate evidence, that probing the consequences of candidate changes before committing one contributes most to reliable convergence; a clarifying question saves one iteration on vague complaints, and attribution has no reliable effect. The loop also runs as a small REST service whose behaviour is byte-identical to the experimental code, and a design for production self-evolution completes the work.

1 Introduction

Test-Time Preference Optimization (TPO; Li et al., ICML 2025, arXiv 2501.12895) improves an answer at inference time without changing any model parameter. The policy samples several candidate answers and a reward model scores them. The policy then compares the best and the worst candidate in a textual critique, which the paper calls a textual loss, and rewrites its answer following the critique, which the paper calls a textual gradient and an update. After D such rounds with N candidates per round, the answer with the highest reward is returned.

This project asks whether the same loop can make an enterprise optimization system respond to its users. Here the "answer" is a plan, the routes of two delivery vehicles, and the feedback is a complaint in plain language such as "vehicle 1 is doing too much work". Task 1 reproduces TPO on its original benchmark. Tasks 2 to 4 build and test the routing version, and Task 5 describes how the corrections could become lasting model improvements in production.

One design decision shapes Tasks 2 to 4. The language model never edits routes. It may only change the optimization model in three ways: add a soft constraint (for example "the load of vehicle 1 should not exceed 92, and every unit above costs a penalty"), change an objective weight, or correct an input value. Gurobi then solves the changed model to optimality, so the hard rules such as vehicle capacity are always respected. Whether a complaint has been resolved is decided by a checking program for each test case, never by the language model. Figure 1 shows the loop. The dashed boxes are the three components built in Task 2, which Task 3 switches on and off.

Task 1 ran on a GPU server with H100 cards. Tasks 2 to 4 ran on a laptop CPU (Apple M2 Pro) with Gurobi 13.0.3 and one language model throughout, `qwen3.8-max` through DashScope with its thinking mode switched off. Before the first model call of each experiment, its code, prompts and test cases were frozen by recording their SHA-256 hashes, and the runners refuse to start if a frozen file has changed. The

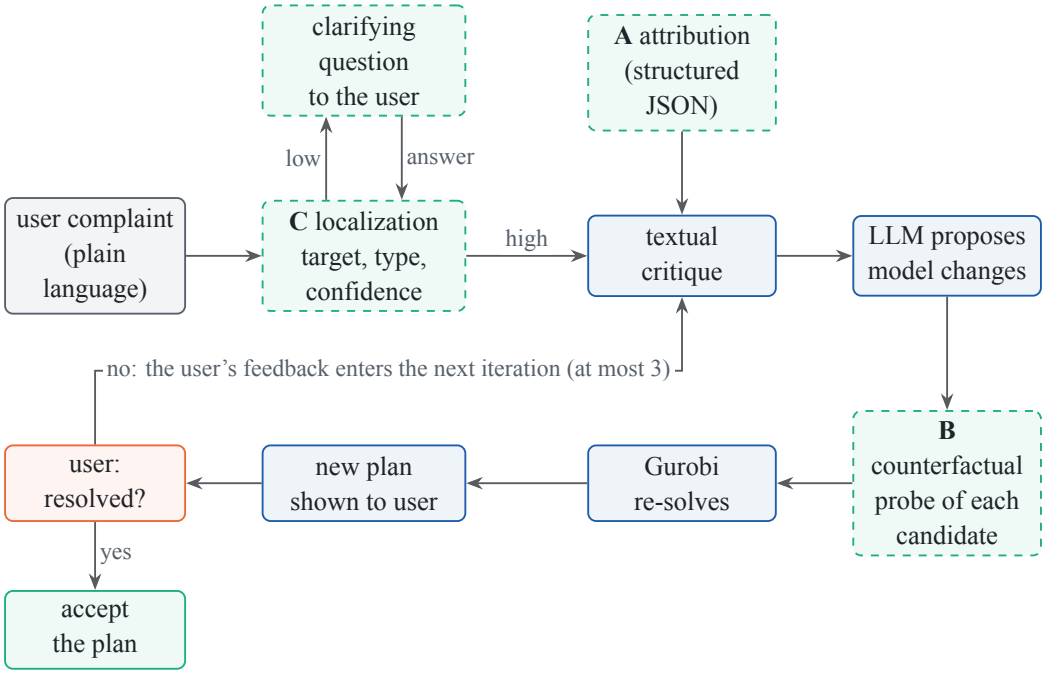


Figure 1. The TPO feedback loop used in Tasks 2 to 4. Solid boxes are fixed steps. Dashed boxes are the mechanisms that the ablation in Task 3 switches on and off: A decision attribution, B counterfactual probing, and C feedback localization with its clarifying question.

results reported here are those of the final frozen versions, **Task 2 v3, Task 3 v3.1 and Task 4 v3.1**. All code, frozen protocols and raw results, including those of earlier runs, are in the submission folder, whose README maps every task to its code and final results.

2 Task 1: Reproducing TPO on AlpacaEval 2

2.1 Setup

The official TPO repository was used with its algorithm, prompts and selection rules unchanged. The policy `allenai/Llama-3.1-Tulu-3-8B-SFT`, a Llama-3.1-8B model with supervised fine-tuning only, wrote the candidates, the critiques and the rewrites and was served by vLLM on one H100. The reward model `FsfairX-LLaMA3-RM-v0.1` ran on a second H100. TPO used $D = 2$ and $N = 5$ on 100 AlpacaEval 2 prompts drawn from the 805 with seed 42, together with their GPT-4 Turbo reference answers. Candidates were sampled with temperature 0.7, top-p 0.95 and at most 2,048 tokens; critiques and rewrites used temperature 0.7, top-p 0.99 and at most 8,192 tokens.

The judge was `qwen3.8-max` instead of GPT-4 Turbo, with AlpacaEval's weighted judging prompt and log-probability parser kept exactly (temperature 1, one output token, thinking off). The win rate (WR) is the mean soft preference for our answer over the reference. The length-controlled win rate (LC) uses AlpacaEval's regression, which was calibrated for GPT-4 and has not been validated for Qwen, so LC is treated as descriptive. Absolute numbers cannot be compared with the paper or the public leaderboard; only changes within a run are meaningful.

Five stages were judged on the same prompts. The *baseline* is the first sampled answer. *Step 0* is the best of the five initial candidates by reward, which measures selection alone. *Steps 1* and *2* are the TPO outputs after one and two rewrites. *Best-of-15* (bon15) keeps the highest-reward answer among 15 samples. It matches the 15 candidates that TPO generates over three rounds, but not the extra compute spent on

critiques. If TPO cannot beat it, the gain may come from sampling more rather than from critique and rewrite.

The experiment was run three times on the same prompts. Run 1 used the repository as provided, with unseeded critique and rewrite requests, and run 2 repeated these settings with a persistent worker pool. Run 3 followed a protocol frozen before it started (`task1_frozen_v1`): every policy request had a fixed seed derived from SHA-256, every request and response was saved, transport retries were limited to three with unchanged parameters, and the analysis, including five paired comparisons, was written before any result was seen. After a three-prompt preflight, exactly one formal run was made, and no parameter was tuned on scores. Run 3 is the main result, and runs 1 and 2 show how stable its conclusions are. Because the runs differ in execution rules as well as in random draws, they are reported side by side rather than averaged.

2.2 Results of the frozen run

Table 1 and Figure 2 give the results of run 3. The reward of the selected answer rises from -4.14 to -0.71. This is guaranteed by construction, because TPO keeps the best answer seen so far; the mean reward of all new candidates in each round also rises, from -4.23 to -2.00 and then -1.53. The judge's win rate rises from 8.77% to 12.09% and 14.13%. With paired bootstrap intervals over prompts (10,000 resamples), step 2 is 5.37 points above the baseline $[+0.74,\ +10.61]$, a reliable gain. Step 1 alone is 3.32 points above the baseline $[-0.78,\ +7.85]$, which is not reliable on 100 prompts.

Table 1. Run 3 (frozen protocol). WR and LC are percentages; SE is the standard error of WR; RM is the mean reward of the selected answers; length is the mean number of characters.

Stage	WR	SE	LC	RM	Length
baseline	8.77	2.60	18.80	−4.14	678
step 1	12.09	3.14	16.84	−1.34	1,343
step 2	**14.13**	3.35	17.57	−0.71	1,629
step 0 (Best-of-5, control)	12.69	3.10	21.51	−2.40	1,024
Best-of-15 (control)	13.97	3.13	**24.97**	−1.59	1,091

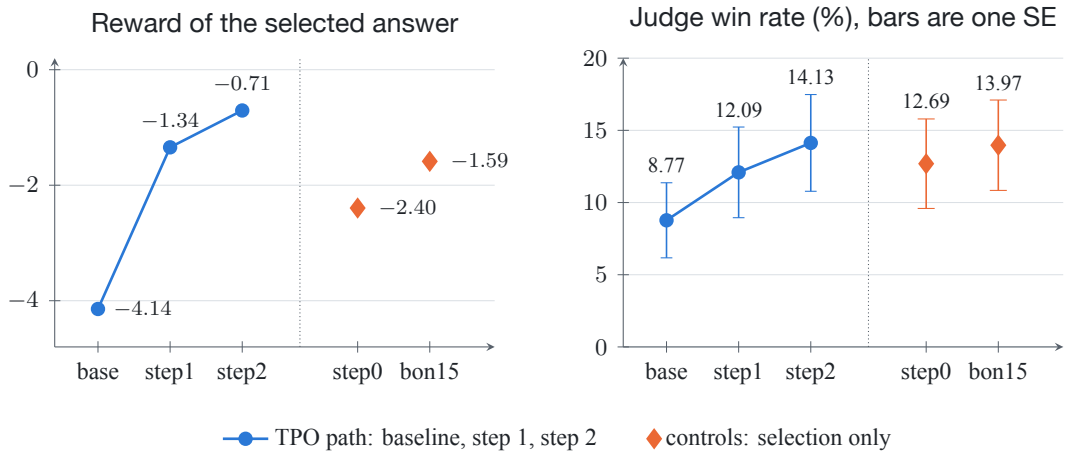


Figure 2. The same answers seen by two evaluators in the frozen run. Left: the reward model prefers every TPO step and rates step 2 far above Best-of-15. Right: the independent judge sees a rising win rate along the TPO path, but step 2 and Best-of-15 are level.

The controls change the reading. Step 2 is only 0.16 points above Best-of-15 $[-5.51, +5.95]$ and 1.45 points above step 0 $[-1.40, +4.89]$, while the reward model rates step 2 well above Best-of-15 ($+0.88$, $[+0.48, +1.28]$). With the same number of candidates, critique and rewriting did no better than sampling more answers and choosing one, and the reward model and the judge disagree on exactly this comparison. The answers also grow longer at every step, from 678 to 1,629 characters on average. LC moves from 18.80% to 16.84% and 17.57%, which is not monotonic, and step 2's LC is 7.40 points below that of Best-of-15 $[-8.28, -6.54]$. A large part of the rise in WR is therefore associated with longer answers. Formatting problems do not explain the result: 3 and 2 of the 500 rewrites in the two rounds lacked the expected tag, and one of the 100 selected step-2 answers reached the length limit.

2.3 All three runs

Table 2. Win rate and length-controlled win rate (percent) in the three runs.

	WR					LC				
Run	base	step1	step2	step0	bon15	base	step1	step2	step0	bon15
1	9.14	10.09	8.27	11.22	14.02	16.29	16.02	10.51	21.67	24.53
2	7.64	9.25	10.56	10.58	12.67	13.46	15.28	14.90	19.67	21.78
3 (frozen)	8.77	12.09	14.13	12.69	13.97	18.80	16.84	17.57	21.51	24.97

Table 2 compares the runs. Four findings hold in all three. The mean reward of the new candidates rises at every step, from between -4.32 and -4.23 in the initial round to between -1.66 and -1.53 after the second rewrite. Step 1 is below step 0 in win rate, so after one rewrite the best answer is judged worse than simply choosing the best of five samples. Step 2 never beats Best-of-15 reliably: it is 5.74 points below in run 1 $[-11.92, +0.01]$, 2.11 points below in run 2 $[-7.67, +3.14]$ and 0.16 points above in run 3. And LC is never monotonic. Whether WR rises monotonically is not stable. It does in runs 2 and 3 but not in run 1, where step 2 is 0.87 points below the baseline, and in run 2 the gain of step 2 over the baseline, 2.91 points $[-1.03, +7.19]$, is not reliable.

2.4 Answer to the task

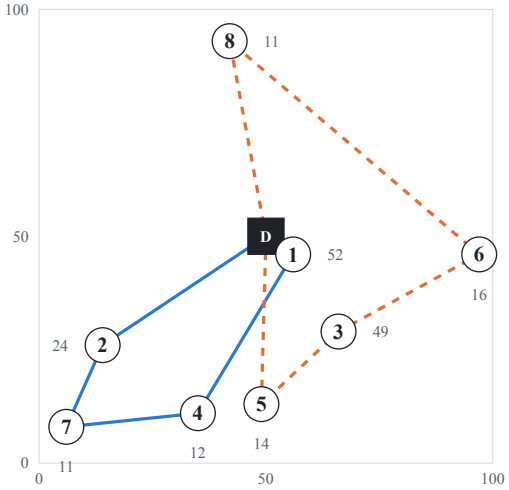
The task asks to confirm that TPO iterations improve the win rate monotonically over the SFT model. This is confirmed only in part. Under the frozen protocol the win rate rises at both steps and the two-step gain is reliable; run 2 shows the same direction without a reliable gain, and run 1 does not show it. LC never rises monotonically, and in no run does TPO beat Best-of-15.

The paper reports no AlpacaEval win rates for an 8B policy. Its Table 1 gives a 70B SFT policy with the same reward model and $D = 2$, $N = 5$, where WR rises from 16.8% to 40.5% and LC from 27.8% to 33.0%. For 8B, its Section 7.4 discusses Llama-3.1-8B-Instruct on HH-RLHF, where the reward falls over iterations. Our policy's reward rises, but the independent judge agrees only in part, so at the 8B scale the gain from TPO is much smaller and less stable than at 70B. The gap between the learned reward and the independent judge is a warning sign for optimizing a learned reward, but it is not proof of reward hacking, since the Qwen judge has its own preferences.

3 Task 2: Explanation, counterfactuals and feedback localization

3.1 The routing instance

The instance is a capacitated vehicle routing problem with one depot, eight customers and two identical vehicles of capacity 100, without time windows (Figure 3). Distances are Euclidean distances rounded to integers, and the objective is the total distance. The instance was drawn with seed 20261022 as the first of 874 random candidates that met five design goals checked by brute force, among them a unique optimum and two customers (1 and 3, with demands 52 and 49) that cannot share a vehicle. The optimal plan is deliberately uneven, which leaves room for realistic complaints about workload, fairness and detours. Tasks 3 and 4 use the same instance.



Node	0	1	2	3	4	5	6	7	8
x	50	56	14	66	35	49	97	6	42
y	50	46	26	29	11	13	46	8	93
Demand	0	52	24	49	12	14	16	11	11

Optimal plan, total distance 351.
Vehicle 1 (solid): 0→1→4→7→2→0, 140 km, load 99.
Vehicle 2 (dashed): 0→5→3→6→8→0, 211 km, load 90.

Figure 3. The instance shared by Tasks 2 to 4 and its optimal plan. Node 0 (D) is the depot; grey numbers are demands. Total demand is 189 against a total capacity of 200. Gurobi and an independent enumerator of all 282,240 solutions agree that 351 is optimal and unique up to swapping the vehicles and reversing a route; the second best plan costs 356.

The model is a three-index mixed integer program with 185 variables and 176 constraints. Its names serve as identifiers in every output: `x[k,i,j]` says that vehicle k drives from i to j; `L[k]`, `W[k]` and `S[k]` are the length, load and number of stops of vehicle k; `A[i]` is the distance driven before customer i is reached; and `cap[k]` is the capacity constraint. Because the vehicles are identical and distances are symmetric, the same plan can be written with the vehicles swapped or a route reversed. Every query therefore fixes one anchor customer to its current vehicle, so that "vehicle 1" keeps a stable meaning, and all ground-truth labels follow the same rule.

3.2 The three components

Decision attribution. Given a decision in the current plan, for example "customers 1 and 3 are on different vehicles", the component forbids it and re-solves with the vehicle labels of the current plan. If the model becomes infeasible, Gurobi's irreducible infeasible subset (IIS) names the hard constraints that force the decision; for customers 1 and 3 these are `cap[1]` and `cap[2]`, because $52 + 49 > 100$. Otherwise the decision is preferred by the objective, and the component reports the extra cost of the best plan without it and the edges that differ. The JSON output also lists capacity slack and the LP duals and reduced costs of the related constraints and variables. A language model writes a short explanation from this JSON, and a checker rejects any explanation that contains a number not in the JSON or names an entity that does not exist.

Counterfactual explanation. Given a change the user asks about, such as moving a customer to the other vehicle or applying a list of model modifications, the component reports feasibility, the change in the objective, and the forced consequences (customers that switch vehicle and edges added or removed), together with the query latency. As the task requires, it does not solve from scratch. It keeps one Gurobi model per state in memory, warm-starts from the current plan and re-optimizes a growing neighbourhood: first only the customers named in the query may change vehicle (L0), then also their two nearest neighbours (L1), then all customers (L2). The neighbourhood is widened when the current level is infeasible, when a soft constraint is still violated, when the plan did not change, or when input data changed. A fresh model solved from scratch serves as the control.

Feedback localization. Given a vague complaint, the language model reads the instance, the current plan with its indicators and the complaint, and returns a structured guess: object, aspect, direction, modification type and threshold. A program maps the guess to a model identifier such as `W[1]`, `A[6]` or `demand[7]` and checks that it exists. The confidence is not reported by the model itself. The model answers five times, and the localization has *high* confidence when at least four answers agree on what the user could confirm (target, direction and limit or corrected value), at least three do not flag ambiguity, and at most one is invalid; otherwise it is *low*. A high-confidence result becomes the structured critique that the TPO loop consumes. A low-confidence result produces only a clarifying question, so nothing reaches TPO until the user has answered.

3.3 Test cases

The ten test cases (Table 3) are shared with Tasks 3 and 4. Each contains a plan, a complaint, a "why" question, a "what if" question, ground truth for all three components, a checker that decides whether the complaint is resolved, a clarification answer for the simulated user, and a reference fix proven to resolve the complaint. Two *control* cases state one clear request. Three *ambiguity* cases are vague, and the most plausible reading of each does not resolve it. Five *side* cases add to a clear request a second condition that the modification language cannot express directly, so that the direct encoding of the request violates the second condition in every optimal plan. The localization targets were labelled by hand. The attribution sets and counterfactual outcomes were computed by a brute-force enumerator that is independent of Gurobi, and every property of a case, such as "the direct fix fails", was asserted by brute force when the case was built.

3.4 Results

Table 4 reports the evaluation required by the task. The language-model parts were repeated three times and gave identical results each time.

Attribution returned the ground-truth set in all ten cases. Two decisions, in U1 and U9, are forced by capacity and were found through the IIS; the other eight are preferred by the objective. For those eight the component and the ground truth share the same definition, so a perfect score mainly shows that the implementation is correct; the IIS cases and the checked explanations carry more information.

The full re-solve agreed with brute force on all 30 queries. The local method agreed on feasibility in every query and on the objective in 25 of 28 feasible ones, but on the change set in only 20 of 28. Its weak point is therefore exactly the information that Task 3's probe relies on. On this instance a full solve takes only about 40 ms, so the local method saves little. On generated instances with 12, 16 and 20 customers and three vehicles (20 queries each), its median latency was 17, 24 and 60 ms against 96 ms, 637 ms and 26.1 s for a full re-solve, while its answer had a higher objective in 3, 13 and 4 of the 20 queries, by at most 9.1%.

Table 3. The ten test cases (task2_v3), with the complaint shortened. Hidden conditions of the ambiguity cases are given to the user simulator only. U9 and U10 start from plans that already carry an earlier modification.

Case	Stratum	Complaint	What resolves it
U1	control	Vehicle 2 drives 211 km; it must not exceed 180 km.	length of vehicle 2 ≤ 180
U2	control	The road between customers 6 and 8 is closed today.	no edge between 6 and 8
U3	ambiguity	"Vehicle 1 is doing too much work."	load of vehicle 1 ≤ 92
U4	ambiguity	A number for customer 7 is wrong; it should be 18.	demand of customer 7 is 18
U5	ambiguity	The drivers are unhappy with how the work is split.	loads differ by ≤ 5
U6	side	Customer 6 within 70 km; vehicle 1 has left and cannot change.	both conditions
U7	side	Vehicle 1 carries at most 94; at most two customers may move.	both conditions
U8	side	Customers 2, 4, 5, 7 on one truck if possible; else 4 and 5 together and at most 370 km in total.	fallback conditions
U9	side	Customers 5 and 7 together; at most two customers move; road 4 to 7 still closed.	all three conditions
U10	side	Road 2 to 7 closed; vehicle 2 has left; 4 and 5 must still share a truck.	all three conditions

Table 4. Task 2 results on the ten test cases. The counterfactual rows compare the local method with a full re-solve on 30 queries per method (the what-if question, the reference fix and the direct encoding of each case). Latencies are mean (median) per query.

Component	Metric	Result
Decision attribution	returned set contains the ground truth	10/10
	precision / recall	1.00 / 1.00
	explanation passes the number and entity check	30/30
Counterfactual, local vs full re-solve	feasibility status agrees	30/30
	objective within 1% (both feasible)	25/28
	change set agrees (both feasible)	20/28
	query latency, local vs full	22.4 (15.2) vs 36.4 (41.8) ms
Feedback localization	target correct, first pass	90%
	correct when predicted high / low	100% / 80%
	cases predicted high / low per run	5 / 5
	correct after the clarifying answer	100%

Localization was correct on the first pass in 90% of cases. Every high-confidence localization was correct, all errors were in low-confidence cases that went to the user as a question first, and after the simulated user's answer all localizations were correct. Half of the cases were rated low: the three ambiguity cases, as intended, and two side cases whose single sentence states two conditions.

4 Task 3: Which component drives convergence

4.1 The loop and the measure

Table 5 shows how TPO maps onto the routing problem. There is no learned reward model. After each commit, a simulated user answers "resolved" or "not resolved" using the case checker; the prompt never sees the checker, its thresholds or the reference fix, which a scan of every prompt sent confirmed. A trial *converges* when, within three TPO iterations, it reaches a plan that is feasible on the real data and meets every condition the user stated, as decided by the checker. This is the measure the task defines.

Table 5. TPO in the paper and in this project.

TPO in the paper	This project
answer being optimized	a list of at most four model modifications, replaced as a whole in each iteration
reward model	none learned; the simulated user's verdict after each commit, and with B the predicted consequences of each candidate before committing
textual loss (critique)	the complaint, plus the attribution JSON (A) or the structured localization (C), the earlier attempts and the user's feedback
textual gradient and update	the language model writes the next complete list, and Gurobi solves the changed model
D iterations, N samples	$D = 3$; three candidates per iteration with B, one without

4.2 Conditions and design

The three mechanisms are switched independently. With **A**, the decisions named in the complaint are attributed on the current plan and the structured result enters the prompt. With **B**, the model writes three candidate lists per iteration, each is evaluated by the counterfactual component, and a selector call sees the predicted consequences and commits one; without B the single proposal is committed directly. With **C**, the complaint is localized first. A low-confidence localization asks the user one question, and if it is still low after the answer, the trial stops and counts as not converged; a high-confidence localization passes the target, modification type and evidence to TPO. Without A and C, only the raw complaint is given. All $2^3 = 8$ combinations were run, together with three control arms. **S1a** and **S1b** are A0B1C0 and A1B1C1 with *blind* selection: the same three candidates and selector, but no predicted consequences. They separate seeing the consequences from drawing more samples. **S2a** is A0B0C0 with the user's clarification appended as plain text whenever the localization asked, which separates the structured fields of C from the extra sentence the user provides.

Every configuration receives the same information from the user. After the first failed plan, the simulated user explains the complaint, and afterwards lists the unmet conditions with the plan's values. Every iteration starts from the plan the user complained about, and the prompt shows the session's earlier modifications. The final batch (v3.1) ran 11 configurations on 10 cases with 5 runs (runs 6 to 10), that is 550 trials with 2,522 model requests. An earlier batch with the same cases and simulated user (v3, runs 1 to 5) ran the eight main configurations without the control arms and without showing earlier modifications; it serves as a replication of the main effects.

4.3 Results

Table 6 gives the final batch. In all eight main configurations, the control and ambiguity cases converged in every trial, so the differences come entirely from the five side cases.

Table 6. Task 3 v3.1, 50 trials per configuration. "Within 3" is the mean and standard deviation over the five runs of the share of the ten cases that converged. Tokens per trial include localization.

Configuration	Within 3 iterations	At iteration 1	Side cases, within 3	Tokens per trial
A0B0C0	96.0% ± 5.5	60%	92%	2.9k
A0B0C1	82.0% ± 10.9	68%	64%	15.1k
A0B1C0	94.0% ± 5.5	64%	88%	11.2k
A0B1C1	86.0% ± 5.5	66%	72%	24.3k
A1B0C0	98.0% ± 4.5	60%	96%	11.1k
A1B0C1	82.0% ± 13.0	70%	64%	22.4k
A1B1C0	98.0% ± 4.5	58%	96%	45.6k
A1B1C1	90.0% ± 7.1	68%	80%	51.6k
S1a (A0B1C0, blind)	86.0% ± 5.5	54%	72%	11.8k
S1b (A1B1C1, blind)	80.0% ± 12.2	62%	60%	55.8k
S2a (A0B0C0 + answer)	88.0% ± 8.4	74%	76%	15.0k

Table 7 gives the effects defined before each batch, for both batches. Intervals are 95% bootstrap intervals clustered by case, and paired tests are exact McNemar tests over (case, run) pairs.

Table 7. Effects in percentage points, convergence within three iterations unless stated otherwise.

Effect	v3 (runs 1 to 5)	v3.1 (runs 6 to 10)
main effects over all cases	A -6.0, B $+6.0$, C -2.0	A $+2.5$, B $+2.5$, C -11.5
B, side cases	$+12$ $[+2,\ +22]$	$+5$ $[-6,\ +17]$
A, side cases	-12 $[-32,\ 0]$	$+5$ $[-1,\ +11]$
probe vs blind with the same 3 candidates, side cases	no blind arm	$+18$ $[+2,\ +34]$, pairs 11 vs 2, $p = 0.023$
blind among 3 vs a single candidate, side cases	no blind arm	-12 $[-22,\ -2]$
C at iteration 1, ambiguity cases	$+61.7$ $[+5,\ +100]$	$+60$ $[0,\ +100]$
C, side cases	-4 $[-16,\ +8]$	-23 $[-43,\ -3]$

4.4 Which mechanism contributes most

The bare loop already converges in 96% of trials in both batches, because the simulated user explains the complaint after a failed plan. The mechanisms therefore have little room, and the pre-registered decision rule, which asks for an interval above zero that is also separated from the second-best mechanism, singles out none of them in either batch. Within that limit, the evidence points to **the counterfactual probe of B as the mechanism that contributes most to reliable convergence, with moderate strength**. On side cases, where the plain fix has consequences the model does not foresee, B's effect was positive in both batches, $+12$ points and reliable in v3, $+5$ points and not reliable in v3.1. The control arms show where the effect comes from. With the same three candidates, choosing after seeing the predicted consequences converged 18 points more often than choosing blind, and choosing blind among three was 12 points worse than taking a single candidate. What helps is seeing the consequences; extra samples without the forecast hurt. The evidence is not stronger because the pooled interval of the probe against blind choice over all cases touches zero ($+9$, $[0,\ +20]$), and over all cases v3.1 gives A and B the same main effect.

C has a different role. On ambiguity cases it raises convergence at the first iteration by about 60 points in both batches, but not within three iterations, because without C the user explains after the first failure. C therefore saves one round of a wrong fix. S2a, which appends the user's answer as plain text, also reaches 100% at the first iteration on these cases, so the gain comes from asking the question, not from the structured fields. C costs reliability on side cases (-23 points in v3.1), because its rule stops a trial when the localization is still unsure after the question.

A has no reliable effect. Its effect on side cases was -12 points in v3 and $+5$ points in v3.1, and it reversed even on case U8, whose prompts are byte-identical in the two batches, so the v3 result was most likely sampling variation. The most economical configuration is the bare loop A0B0C0 with 2.9k tokens per trial; A or B alone costs about four times as many tokens, and C about five times, mostly for the five localization samples.

5 Task 4: A minimal feedback collection pipeline

5.1 Architecture

The loop runs as a FastAPI service that stores everything in JSON files. Table 8 maps the four required components to the implementation, and Figure 4 shows the structure. All solver and language-model work runs on a single engine thread that takes jobs from a queue in arrival order, because a Gurobi environment and the resident models of the counterfactual probe are not thread-safe; the HTTP threads only validate, store, enqueue and wait. Every file is written to a temporary name and then renamed, so a crash cannot leave a half-written record, and the service reloads all sessions after a restart. A restart test (SIGTERM while a complaint awaited the user's reply) confirmed that sessions resume.

Table 8. Required components and their implementation (`Task 4/task4/`).

Required component	Implementation
feedback ingestion endpoint	`POST /sessions/{sid}/feedback` stores the complaint text with the session ID and a snapshot of the plan it refers to (version and routes). A complaint about a plan that is no longer current is rejected with HTTP 409.
complaint processor	Task 2's localization turns the stored complaint into the structured critique. A low-confidence result returns a clarifying question instead, answered through `POST /feedback/{fid}/clarification`, and never reaches TPO.
trigger mechanism	receiving the complaint (or the answer to the question) enqueues a TPO refinement run; by default the request waits and returns the refined plan. `POST /feedback/{fid}/reply` accepts the plan or starts the next iteration with the user's comment.
logging layer	`iterations.jsonl` holds one record per iteration with the input plan, the complaint and replies so far, the critique, the candidates and probe results, the committed modifications, the refined plan and its feasibility: the solver status and an independent check on both the effective data and the data the user confirmed.
storage	JSON files with atomic writes; `events.jsonl` records every API action and status change, so a session can be replayed.

5.2 Rules and equivalence with Task 3

A low-confidence localization never enters TPO: the user is asked first, and if the complaint is still unclear, the service asks the user to rephrase. A refined plan stays a proposal until the user replies that the complaint

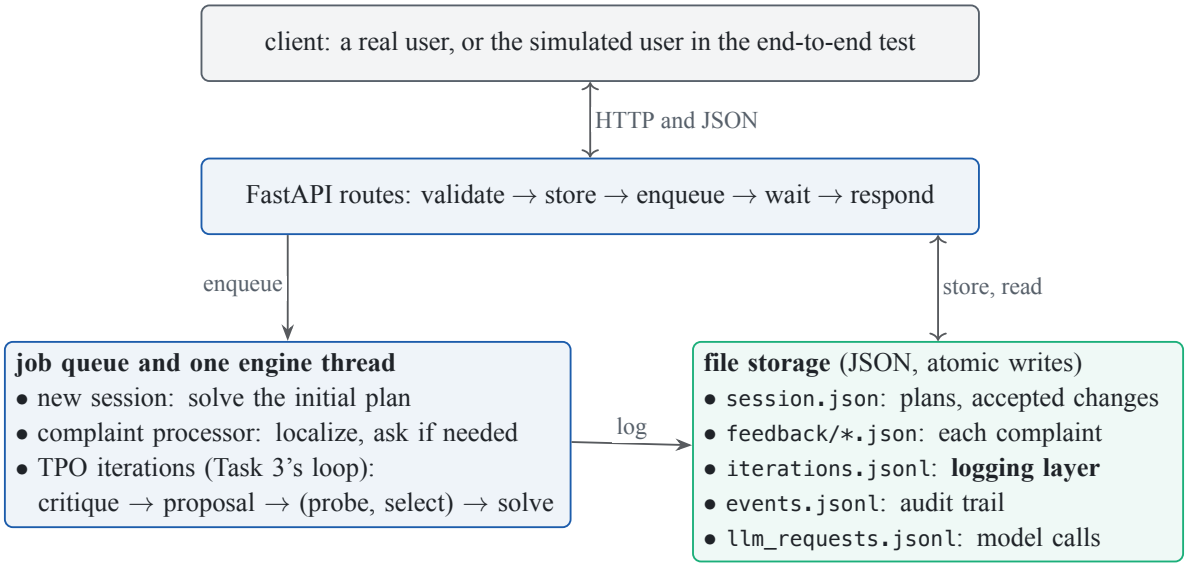


Figure 4. Structure of the Task 4 service. HTTP threads never touch the solver or the language model; one engine thread does all of that work and writes the logs.

is resolved; any other outcome returns the session to the plan that was complained about, while the logs keep everything. The user is only shown plans they have not seen before, a session handles one complaint at a time against a named plan version, and an unreachable language model marks the complaint `failed` instead of being read as low confidence. Each session chooses one of Task 3's eight configurations. The default A0B0C1 follows the task, which makes the structured localization the critique and requires low-confidence cases to ask first; Task 3 suggests also switching B on for complaints with side conditions.

The service does not reimplement the loop. It imports Task 3's frozen code read-only and refuses to start if the hash of an imported file has changed. Equivalence tests drive the service over HTTP with Task 3's simulated user and compare each iteration with a direct call of the loop. For all eight configurations and scenarios ranging from a first-iteration fix to a failure after three iterations and a clarification, the prompts, candidates, selection, committed modifications and plans are byte-identical. All 148 automated tests pass without network access.

5.3 End-to-end run

The final service was frozen and run once on the ten cases with all mechanisms on (A1B1C1), using a real server process, real HTTP requests, the real language model and Gurobi. The client was Task 3's simulated user, which sends only what a user would send: the plan, the complaint, the answer to a question and its replies. The service resolved 8 of 10 complaints, 7 of them at the first iteration, and asked a clarifying question for 4. The median time of one complaint request was 24.9 s, almost all of it language-model time, since a full Gurobi solve takes about 40 ms. The two unresolved cases are hard in Task 3 as well: for U9 the localization was still unsure after the question, so by rule nothing reached TPO, and for U10 three iterations did not meet all conditions. All sessions were closed at the end, and every iteration record has all required fields.

A short exchange from this run illustrates the pipeline. For case U3 the user wrote "Vehicle 1 is doing too much work." The five localization samples disagreed, so the service asked whether vehicle 1's load (99 units), route length (140 km) or number of stops (4) was meant. The user answered that the load should be at most 92 units. The critique named the target `W[1]` with the evidence `W[1] = 99` and the limit 92, the committed modification was a soft constraint $W_1 \leq 92$ with penalty 100, and Gurobi returned a plan

of 390 km in which vehicle 1 carries 91 units. The user accepted it, although it is 22 km longer than the shortest plan that meets the condition; Task 5 treats this kind of collateral cost explicitly. The service is deliberately minimal, as the task allows: complaints from different sessions wait for the single engine thread, and there is no authentication, tenancy or rate limiting.

6 Task 5: Production-grade self-evolution

TPO corrects answers at inference time but leaves the model unchanged, so a correction made today must be made again tomorrow. This section summarizes a plan for turning corrections into lasting improvements; the full design is `Task 5/task5/DESIGN.md`. The plan keeps the division of labour of Tasks 2 to 4: the solver owns routes and feasibility, and the user's reply after verification decides whether a complaint is resolved. What can evolve are the three decisions of the language model: how to localize a complaint, whether to ask first, and which modifications to propose (Figure 5). Thresholds and frequencies below are proposed starting values.

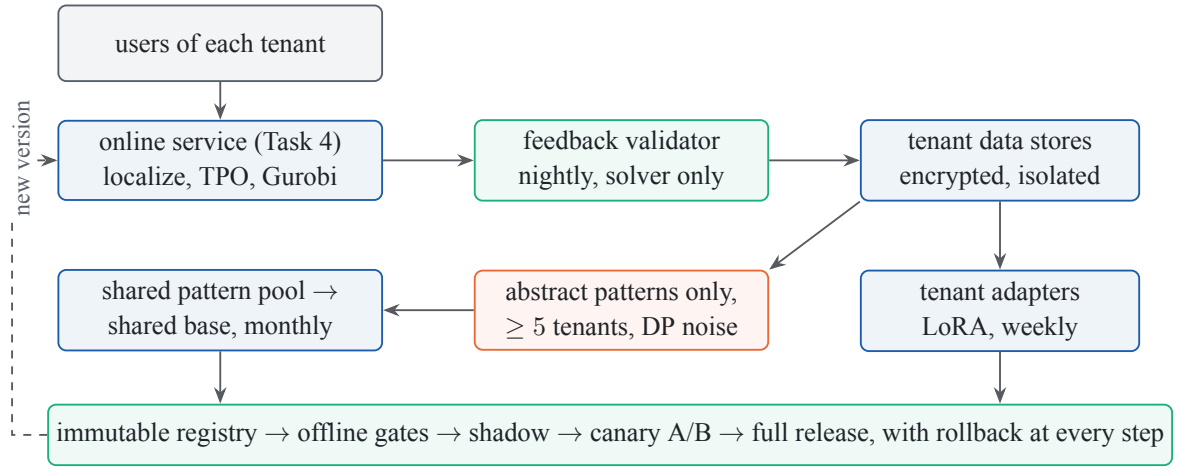


Figure 5. Architecture for self-evolution. The service records each complaint thread with its model, prompt and configuration versions; a solver-based validator labels the threads; each tenant's data trains its own adapter; only abstract patterns reach the shared base; and every new version passes offline, shadow and canary stages before full release.

6.1 From correction to evolution

Each complaint thread in the Task 4 logs is a complete trajectory, from the complaint and the clarifying exchange to the candidates with their predicted consequences, the committed modifications, the plans and the outcome. For supervised fine-tuning of the proposer, the input is the first-iteration prompt of a resolved and verified thread and the target is the modification list finally accepted, so that a fix which needed three iterations is learned as a first answer. For the localizer, the target is the final localization, or the clarifying question itself when the user's answer changed the target, which teaches the model when to ask. Pairs for direct preference optimization (DPO) come from accepted against rejected lists, kept only when the validator confirms that the rejected list really failed, and from probe-labelled or solver-labelled candidates, kept only after a full re-solve confirms the label. If a reinforcement step is used, its reward is the verified condition minus the collateral cost; user acceptance only decides which threads count. The proposed cadence is nightly validation without the language model, labels that become final seven days after acceptance, weekly per-tenant adapters, a monthly shared base, and about 30% replay of older data. Two results of this project shape the plan. The confidence gate that decides whether to ask stays a rule

outside the trained model, because a localizer trained on acceptance could learn never to ask; and since Task 3 found that the probe's forecast helps, the served loop keeps the probe.

6.2 Feedback quality

User feedback is noisy: users may praise a plan they misunderstood or complain about a plan that does what they asked. The Meta RLUF study (Han et al., arXiv 2505.14946, 2025) shows the risk of optimizing such a signal directly. A reward model for the rare "Love" reaction raised the Love rate by 28% online, but the model learned shortcuts: answers containing "bye" rose from 0.72% to 2.8%, follow-up questions fell by about half, and helpfulness fell from 1.53 to 1.28. Here the counterpart is the acceptance rate. A model could stop asking clarifying questions, please one user at another driver's cost, or make a complaint disappear by changing data the user never mentioned. The plan therefore makes the clarification rate on ambiguous complaints a release gate, puts collateral cost into the reward, and requires feasibility on data the user has confirmed. Before training, hard filters remove system failures, test traffic and threads from rolled-back versions. The verifier re-checks the localized condition on the accepted plan, and threads where user and verifier disagree go to human review instead of training. Each user's reliability is a Beta posterior of their agreement with the verifier, and no user may contribute more than 5% of a tenant's weekly examples.

The task asks how the components of Task 2 can flag two kinds of bad feedback. A small validator that uses only the solver, attribution and counterfactuals was built and run on sessions from the Task 4 run (Table 9). All ten scenarios, six checks, three controls and one constructed case, received the expected verdict; the feedback texts are illustrative, so this demonstrates the mechanism rather than measuring a detection rate.

Table 9. The feedback validator on sessions from the Task 4 run (`Task 5/task5/validator/`).

Case	Feedback	What the components show	Verdict
V4	"Customer 3 now gets its delivery early."	measured change: the distance before customer 3 went from 60 to 129 km	praise excluded
V3	"Thanks for putting customers 3 and 5 on the same truck."	they shared a truck before the fix; attribution on the old plan shows the objective preferred it	praise excluded
V5	"Good, the second driver has fewer stops now."	true, but on the confirmed data vehicle 1 carries 124 units against a capacity of 100	incident
V1	"Vehicle 1 now drives 235 km." (the user earlier imposed a load limit of 92)	counterfactual without that input: 140 km, total distance 390 → 351	self-imposed; show the trade-off
V2	"Customers 2 and 5 should share a truck." (the user entered a demand of 54)	IIS: separated by `cap[1]` and `cap[2]`; with the user's value reverted, only preferred by the objective	caused by the user's data

A complaint against the user's own constraint is not evidence that the model erred, so it must not create a negative example; it should instead teach the response "this conflicts with your earlier request, and this is what each choice costs".

6.3 Multi-tenant isolation, A/B testing and rollback

Each tenant has separate storage and encryption keys, its own LoRA adapters on a frozen shared base, and few-shot examples from its own data only. Only abstract pattern records leave a tenant, such as the target type (`W[k]` rather than a named vehicle), the modification type and the relative threshold; they are

replayed on synthetic instances, pooled only after appearing in at least five tenants, and released with differential-privacy noise. Before a shared base is released, canary strings planted in tenant data must never be generated and membership-inference tests must stay at chance level.

A new version first passes offline gates, including Task 3's harness on the frozen test cases, and then runs in shadow mode for one to two weeks, where its proposals are solved and checked but never shown. It is then released as a canary, randomized by complaint thread and stratified by tenant, at 5%, 25%, 50% and 100% of traffic. Detecting a drop in acceptance from 95% to 93% at the 5% level with 80% power needs about 2,200 threads per arm. A plan shown to a user that is infeasible on confirmed data triggers an immediate rollback. A statistical rollback follows when the upper confidence bound of the acceptance difference falls below -2 points, when repeated complaints rise by more than 2 points, when collateral distance rises by more than 5%, or when the clarification rate on ambiguous complaints falls by more than 30% even if acceptance rises, because that is the RLUF pattern. Every version is immutable, so a rollback is a routing change, and data from a rolled-back version is kept out of training.

7 Verification and limitations

All results were checked again on the submitted folder. The four test suites of Tasks 2 to 5 (186, 278, 148 and 6 tests) run without network access, and all 618 pass. The scripts that recompute the metrics of Task 2 v3, Task 3 v3.1, Task 4 v3.1 and the Task 5 validator from the saved raw results were run again, and every regenerated result file is byte-identical to the one reported. For Task 1, every WR, SE, LC and mean length of all three runs was recomputed from the saved judge annotations, and the paired bootstrap intervals of runs 2 and 3 were reproduced exactly.

The main limitation of Tasks 2 to 4 is size. With one nine-node instance and ten test cases, one case moves a rate by ten points, and effects within one stratum changed by more than ten points between the two batches of Task 3, so its answer carries moderate rather than strong evidence. The users in Tasks 3 and 4 are simulated and answer more consistently than people would. The language model `qwen3.8-max` is an alias without a fixed snapshot, and fixed seeds did not reproduce its outputs: with identical prompts and seeds in the two batches, only one of 120 first-iteration outputs was the same. In Task 1 the judge is Qwen rather than GPT-4 Turbo, LC keeps a calibration fitted for GPT-4, the policy has 8B parameters against 70B for the paper's main AlpacaEval result, and only 100 prompts were used.

8 Conclusion

On the public benchmark, TPO made the reward model steadily more satisfied, and under a frozen protocol the independent judge's win rate also rose reliably over two iterations. The gain did not survive length control, did not exceed simply sampling fifteen answers and keeping the best, and was not stable across three runs. At the 8B scale, TPO's critique and rewrite added little beyond selection.

In the routing feedback loop, letting the language model change only the optimization model, and leaving routes and feasibility to the solver, gave a system whose every answer could be checked. Probing the consequences of a change before committing it helped most when the obvious fix had side effects; asking a clarifying question resolved vague complaints one iteration earlier; and attribution did not help, because knowing why the plan looks as it does does not tell the model what the user means.

A common thread runs through the project: any proxy, whether a learned reward or a user's approval, can drift away from the quality it stands for. The production design therefore treats user approval as a gate and builds its training signal on facts that the solver can verify.